

HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Ciberseguridad Avanzada | Curso 2026

Práctica · Práctica 1 — Red Team

EQUIPO DE DESARROLLO

P1	Ivan Medina Castro	Frontend — React + Vite + Tailwind
P2	Macarena Rogerio	Backend — FastAPI + Python
P3	Nacho García Monge	Playwright — Automatización
P4	Carlos Bañuelos Fernández	DevTools — Chrome DevTools Protocol
P5	Jose María López Ausín	IA + GitHub — Claude API

1.	Resumen ejecutivo	4
2.	Descripción del problema y justificación	5
2.1.	El ecosistema actual de herramientas de auditoría web	5
2.2.	La propuesta de HookSuite	5
2.3.	Relevancia en el contexto académico	6
3.	Arquitectura técnica	7
3.1.	Visión general del sistema	7
3.2.	Proceso de decisión arquitectónica	7
3.3.	Stack tecnológico	8
3.4.	Contratos de integración activos — Práctica 1	9
4.	Proceso de desarrollo	10
4.1.	Módulo Frontend (P1 — Ivan Medina Castro)	10
4.1.1.	El módulo	10
4.1.2.	Proceso de desarrollo	11
4.1.2.1.	Fase 0 — Investigación y decisiones de arquitectura	11
4.1.2.2.	Fase 1 — Construcción inicial con mock mode y proxy PAC	11
4.1.2.3.	Fase 2 — Cambio de arquitectura	12
4.1.2.4.	Fase 3 — Integración con el Backend real	12
4.1.2.5.	Fase 4 — Últimos ajustes	13
4.2.	Módulo Backend (P2 — Macarena Rogerio)	13
4.2.1.	El módulo	13
4.2.2.	Proceso de desarrollo	14
4.2.2.1.	Fase 1 — Construcción del servidor base	14
4.2.2.2.	Fase 2 — Arquitectura inicial con mitmproxy e integración en producción	16
4.2.2.3.	Fase 3 — El pivote	18
4.2.2.4.	Fase 4 — Nueva arquitectura: cliente persistente y módulos de auditoría	18
4.2.2.5.	Fase 5 — Últimos ajustes y estado final	19
4.3.	Módulo Playwright (P3 — Nacho García Monge)	19
4.3.1.	El módulo	19
4.3.2.	Proceso de desarrollo	21
4.3.2.1.	Fase 1 — Setup y optimización de velocidad	21
4.3.2.2.	Fase 2 — Reconocimiento automático	21
4.3.2.3.	Fase 3 — Automatización de ataques	21

4.3.2.4. Fase 4 — El pivote y sus consecuencias para P3	22
4.4. Módulo DevTools (P4 — Carlos Bañuelos Fernández)	22
4.4.1. El módulo	22
4.4.2. Proceso de desarrollo	24
4.4.2.1. Fase 1 — Setup y construcción del módulo	24
4.4.2.2. Fase 2 — Validación contra DVWA	24
4.4.2.3. Fase 3 — El pivote y sus consecuencias para P4	25
4.5. Módulo IA (P5 — Jose María López Ausín)	26
4.5.1. El módulo	26
4.5.2. Proceso de desarrollo	27
4.5.2.1. Fase 1 — Construcción del módulo	27
4.5.2.2. Fase 2 — Orquestador y modo mock	28
4.5.2.3. Fase 3 — El pivote y sus consecuencias para P5	28
5. Guía de despliegue	29
6. Manual de uso	30
7. Conclusiones y lecciones aprendidas	31
7.1. Lo que funcionó mejor de lo esperado	31
7.2. Lo que fue más difícil de lo previsto	31
7.3. Qué haríamos diferente si empezáramos de nuevo	31
7.4. Aprendizajes sobre integración de sistemas complejos	32
8. Road map de mejora para la Práctica 2	33

SECCIÓN 1

Resumen ejecutivo

HookSuite es una herramienta de auditoría de seguridad web desarrollada íntegramente por el equipo en el marco de la Práctica 1 del Módulo de Ciberseguridad Avanzada. Su objetivo es automatizar el proceso de detección de vulnerabilidades web combinando análisis de tráfico HTTP en tiempo real, fuzzing automatizado de parámetros y análisis inteligente mediante inteligencia artificial.

Funcionalmente, HookSuite opera de forma similar a Burp Suite pero con tres diferencias clave: está construida desde cero, es accesible desde cualquier navegador sin instalación de software adicional en el cliente, e incorpora inteligencia artificial para clasificar y priorizar vulnerabilidades según el estándar OWASP.

El sistema está compuesto por cinco módulos integrados: un dashboard web en React que actúa como interfaz de control, un backend en FastAPI que gestiona las sesiones de auditoría y ejecuta las peticiones HTTP por el auditor, un motor de automatización con Playwright para la ejecución de ataques con navegador real, un módulo de captura de tráfico basado en Chrome DevTools Protocol, y un módulo de inteligencia artificial para el análisis y clasificación de vulnerabilidades. Los módulos de Playwright, DevTools e IA están integrados en la arquitectura del sistema y su activación completa está planificada para la Práctica 2.

La herramienta está desplegada en un servidor Hetzner Cloud CX22 y es accesible desde cualquier navegador a través de su URL pública. En su estado actual, el Spider, el Repeater, el Intruder y las Utilidades están completamente operativos y han sido validados contra DVWA (Damn Vulnerable Web Application). El desarrollo completo, incluyendo el historial de commits, está disponible en el repositorio público de la organización Feet-Lovers en GitHub.

SECCIÓN 2

Descripción del problema y justificación

El ecosistema actual de herramientas de auditoría web

Las herramientas de auditoría de seguridad web más utilizadas en la industria —Burp Suite, OWASP ZAP, Nikto— comparten un patrón común: requieren instalación local, generan grandes volúmenes de tráfico fácilmente detectable, y delegan en el analista la mayor parte del trabajo de interpretación de resultados. Son herramientas potentes pero con tres limitaciones estructurales que HookSuite aborda directamente.

La primera es la dependencia del analista humano. Herramientas como Burp Suite interceptan el tráfico y presentan los datos en bruto, pero la interpretación —determinar qué es una vulnerabilidad, qué severidad tiene, qué ataque ejecutar a continuación— recae íntegramente en el criterio del analista. Esto convierte la auditoría en un proceso que escala mal: más tráfico equivale a más tiempo de análisis manual.

La segunda es la accesibilidad. La mayoría de herramientas profesionales requieren configuración local del sistema operativo, instalación de certificados de seguridad y conocimientos técnicos previos para operar. Esto eleva la barrera de entrada y dificulta su uso en entornos educativos o en equipos con perfiles heterogéneos.

La tercera es la trazabilidad. Las auditorías realizadas con herramientas tradicionales generan logs dispersos, difícilmente exportables y sin estructura estandarizada. Integrar los resultados en un flujo de trabajo moderno —CI/CD, ticketing, informes automáticos— requiere desarrollo adicional específico para cada herramienta.

La propuesta de HookSuite

HookSuite nace como respuesta a estas tres limitaciones. Su arquitectura combina tecnologías modernas para construir una solución que es a la vez accesible, inteligente y trazable.

La accesibilidad se resuelve mediante un dashboard web accesible desde cualquier navegador sin instalación ni configuración adicional. El analista dirige el proceso —introduce la URL objetivo, define el scope, lanza el spider, selecciona peticiones para el Repeater y configura los ataques en el Intruder— mientras el servidor ejecuta las operaciones pesadas sin intervención adicional del cliente.

La automatización del análisis se resuelve mediante un módulo de inteligencia artificial integrado en el sistema. Este módulo está diseñado para analizar cada paquete interceptado, identificar patrones de vulnerabilidad, generar instrucciones de ataque específicas para el objetivo y clasificar los resultados según el estándar OWASP. El analista recibirá vulnerabilidades clasificadas y priorizadas, no datos en bruto.

La trazabilidad se resuelve mediante un sistema de registro estructurado. Cada vulnerabilidad detectada genera una ficha completa con identificador único, tipo, severidad, URL afectada, payload utilizado y recomendación de mitigación, exportable en formato JSON estándar.

| Relevancia en el contexto académico

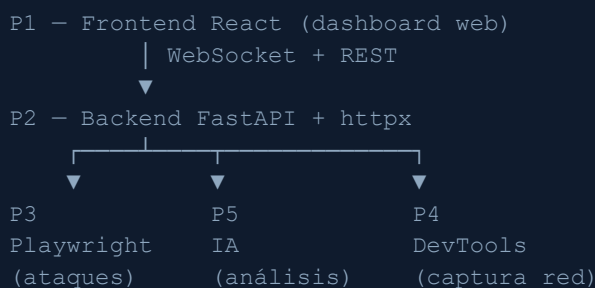
Este proyecto desarrolla competencias en cinco áreas simultáneamente: desarrollo web full-stack, seguridad ofensiva, integración de APIs de inteligencia artificial, trabajo con protocolos HTTP a bajo nivel e infraestructura en la nube. La naturaleza distribuida del sistema —cinco módulos integrados desarrollados en paralelo por cinco personas— añade una dimensión de ingeniería de software que va más allá del ejercicio técnico individual.

SECCIÓN 3

Arquitectura técnica

Visión general del sistema

HookSuite está compuesto por cinco módulos independientes que se comunican a través de una capa de backend centralizada. Cada módulo fue desarrollado por un miembro del equipo de forma autónoma y se integra con el resto a través de contratos de API definidos previamente.



Los módulos P3, P4 y P5 están integrados en la arquitectura y su activación completa está planificada para la Práctica 2.

Proceso de decisión arquitectónica

El diseño de HookSuite partió de una pregunta técnica concreta: ¿cómo construir una herramienta de auditoría web accesible desde el navegador sin instalar software adicional en el cliente?

La investigación inicial identificó tres limitaciones fundamentales del navegador que condicionaron toda la arquitectura. El modelo de seguridad del navegador impide interceptar el tráfico de otras pestañas y aplicaciones por la política de mismo origen, actuar como proxy TCP al no tener acceso a sockets TCP arbitrarios, y modificar headers protegidos como Host, Origin o Cookie en las peticiones fetch y XHR. La conclusión fue que la función central de una herramienta de auditoría es técnicamente imposible si toda la lógica reside en el frontend. La solución pasaba por mover el proxy al servidor.

Con esa premisa, el equipo evaluó cinco opciones para gestionar la interceptación del tráfico:

Opción	Resultado
--------	-----------

Extensión de navegador	✗ Descartada — rompe el concepto de web app
Certificado CA propio	✗ Descartada — requiere instalar certificado en el cliente
Proxy SOCKS5	✗ Descartada — configuración manual compleja
Electron híbrido	✗ Descartada — requiere instalar aplicación de escritorio
Archivo PAC + Web-Sockets	✓ Seleccionada

La opción seleccionada combinaba dos elementos. Un archivo PAC (Proxy Auto-Configuration) alojado en el servidor — un fichero JavaScript que el navegador lee antes de cada petición para decidir si enrutar el tráfico a través del proxy. Y WebSockets como canal de comunicación bidireccional en tiempo real entre el frontend y el backend. Esta combinación ofrecía compatibilidad con todos los navegadores modernos, sin instalaciones en el cliente, y soporte nativo para múltiples usuarios simultáneos mediante tokens de sesión UUID.

La validez de esta arquitectura fue confirmada por Caído — una herramienta de seguridad web profesional con arquitectura cliente-servidor idéntica que también requiere configuración manual del proxy, lo que confirmó que no existe ninguna alternativa técnica viable que evite ese paso sin instalar una extensión o aplicación.

Esta arquitectura inicial fue posteriormente descartada cuando al exponerla al exterior los bots saturaron el servidor. El equipo pivotó hacia un modelo donde el servidor realiza las peticiones HTTP directamente por el auditor — eliminando la necesidad de configuración del proxy en el navegador y resolviendo el problema de saturación. Esta decisión redefinió el rol de cada módulo y condicionó la fase de integración del proyecto.

Stack tecnológico

Módulo	Tecnologías	Responsable
Frontend	React 19 + Vite + Tailwind CSS	P1 — Ivan
Backend	Python 3.11 + FastAPI + httpx + WebSockets	P2 — Macarena
Playwright	Python + Playwright + httpx	P3 — Nacho
DevTools	Python + WebSockets + httpx	P4 — Carlos
IA	Claude Code	P5 — Jose María
Infraestructura	Docker + Hetzner Cloud CX22 + Nginx	Todos

Contratos de integración activos — Práctica 1

Integración	Canal	Estado
P1 ↔ P2 (Frontend — Backend)	WebSocket + REST	☒ Activo

Los contratos de integración de P3, P4 y P5 con el Backend se definirán y documentarán durante la Práctica 2.

SECCIÓN 4

Proceso de desarrollo

| Módulo Frontend (P1 — Ivan Medina Castro)

El módulo

El Frontend es la interfaz visual de HookSuite — un dashboard web accesible desde cualquier navegador sin instalación adicional. Construido sobre React 19, Vite y Tailwind CSS, con JetBrains Mono como tipografía de código y Syne para la interfaz, se comunica con el Backend exclusivamente mediante WebSocket para recibir eventos en tiempo real y REST para enviar las acciones del auditor. La gestión del estado compartido entre paneles se centraliza en un contexto global que mantiene la conexión WebSocket activa, el identificador de sesión UUID del auditor y los datos que fluyen entre los distintos módulos de la interfaz.

El Frontend se organiza en seis paneles accesibles desde el sidebar:

El **panel Proxy** es el centro de operaciones de la auditoría. El auditor introduce la URL objetivo, selecciona la velocidad del análisis — rápido, normal o completo, que determina el número máximo de páginas que el spider visitará — y lanza el proceso. Las peticiones que el servidor realiza por el auditor aparecen en tiempo real agrupadas por URL. Los formularios detectados en cada página se muestran como subelementos desplegados bajo su URL correspondiente, lo que permite identificar de un vistazo los vectores de ataque disponibles. El panel de estado muestra las cookies de sesión activas en verde cuando el auditor está autenticado, y ofrece tres acciones: liberar la sesión activa sin perder el historial de peticiones, limpiar el panel manteniendo la sesión, o iniciar una nueva auditoría completa reseteando todo el estado.

El **Repeater** permite modificar y reenviar cualquier petición manualmente. El auditor puede enviar al Repeater cualquier petición interceptada en el panel Proxy con un solo clic, y desde ahí modificar el método HTTP, la URL, los headers y el body antes de reenviarla. La respuesta se muestra en cuatro vistas: Raw muestra la respuesta tal como llega del servidor; Pretty formatea automáticamente el JSON para facilitar su lectura; Preview renderiza el HTML de la respuesta en un iframe reescribiendo las URLs relativas para que los recursos del objetivo se carguen correctamente; y Headers muestra los headers de respuesta con las cookies resaltadas en verde para identificarlas fácilmente.

El **Intruder** automatiza el fuzzing de parámetros al estilo Burp Suite. Al recibir una petición detecta automáticamente todos los parámetros GET y POST presentes. El auditor selecciona el parámetro que quiere atacar marcándolo con el símbolo * — el marcado se resalta en naranja en tiempo real tanto en la URL como en el body. El sistema sustituye ese marcador por cada payload de la lista seleccionada y envía todas las peticiones de forma

automatizada. Soporta cuatro tipos de ataque con sus respectivas listas de payloads: SQL Injection, Blind SQLi, XSS y fuzzing genérico. Los resultados se muestran en una tabla en tiempo real donde las peticiones que reciben una respuesta identificada como vulnerable se marcan en rojo.

Las **Utilidades** agrupan cuatro herramientas auxiliares de uso frecuente en auditorías web. El Encoder/Decoder transforma texto entre los formatos más comunes — Base64, URL encoding, HTML encoding y decodificación de tokens JWT. El Hash Generator calcula los hashes MD5, SHA1, SHA256 y SHA512 de cualquier texto. El Regex Tester permite probar expresiones regulares contra texto de prueba con resaltado visual de los matches en tiempo real. El Payload Generator organiza colecciones de payloads por tipo de ataque — SQLi, Blind SQLi, XSS y fuzzing genérico — con opción de copiar payloads individuales o la lista completa.

El **panel Vulnerabilidades** y el **panel Red** están completamente contruidos e integrados en la interfaz. El panel Vulnerabilidades está diseñado para recibir las detecciones del módulo de IA clasificadas por severidad — crítica, alta, media y baja — con descripción de la vulnerabilidad, payload utilizado y recomendación de mitigación. El panel Red está diseñado para mostrar el tráfico capturado por el módulo DevTools en tiempo real, con código de colores para identificar peticiones limpias, sospechosas y vulnerables. Ambos paneles permanecen inactivos en esta entrega porque los módulos que los alimentan están pendientes de integración completa en la Práctica 2.

Proceso de desarrollo

4.1.2.1. Fase 0 — Investigación y decisiones de arquitectura

Antes de que Ivan iniciara el desarrollo, el equipo realizó una investigación técnica para definir la arquitectura de la herramienta. La conclusión fue que HookSuite operaría como un proxy en el navegador del usuario — el tráfico del auditor pasaría a través del servidor antes de llegar al objetivo, permitiendo interceptarlo y analizarlo. Esta arquitectura requería que el usuario configurara su navegador apuntando a un archivo PAC que el servidor generaba dinámicamente.

Para minimizar la fricción de esa configuración, el equipo diseñó un asistente de onboarding que detectaba automáticamente el navegador y sistema operativo del usuario y mostraba instrucciones paso a paso personalizadas. El asistente verificaba cada dos segundos si el proxy estaba activo y cerraba el modal automáticamente cuando lo detectaba. Como alternativa para usuarios que no quisieran configurar el proxy, se diseñó también un importador de peticiones que permitía pegar peticiones en formato raw HTTP o cURL directamente en el Repeater.

Estas decisiones definieron el alcance inicial del módulo de Ivan y los componentes que debía construir.

4.1.2.2. Fase 1 — Construcción inicial con mock mode y proxy PAC

Ivan construyó el Frontend completo siguiendo su manual de desarrollo, entregando la primera versión funcional con toda la estructura base del proyecto: layout con sidebar de

navegación, sistema de componentes UI reutilizables, hooks de WebSocket y sesión, las seis páginas principales, el asistente de configuración del proxy con detección automática de navegador y sistema operativo, el importador de peticiones, y un sistema completo de mock data.

El sistema de mock data fue una de las decisiones técnicas más relevantes del módulo. Dado que el Frontend se desarrollaba en paralelo al Backend, Ivan construyó un conjunto de datos simulados que replicaban exactamente el formato que emitiría el WebSocket real — peticiones interceptadas, vulnerabilidades detectadas, paquetes de red. Esto permitió desarrollar y validar toda la interfaz de forma completamente independiente, sin bloqueos por dependencias entre módulos. La transición posterior al Backend real fue más controlada al tener ya definido un contrato de datos concreto.

4.1.2.3. Fase 2 — Cambio de arquitectura

Al desplegar el sistema en el servidor de producción y exponerlo al exterior, los bots saturaron el proxy HTTP. El tráfico automatizado externo colapsó el servidor impidiendo su uso normal. El equipo tomó la decisión de cambiar la arquitectura completamente: en lugar de interceptar el tráfico del navegador del usuario, el servidor realizaría las peticiones HTTP directamente por el auditor usando un cliente HTTP propio.

Este cambio tuvo un impacto directo y significativo en el Frontend. El asistente de configuración del proxy quedó en el código pero sin uso activo en producción. El modelo de interacción cambió completamente — de pasivo, donde el auditor navegaba y el sistema interceptaba su tráfico, a activo, donde el auditor introduce una URL y el servidor la explora. El panel Proxy pasó de ser un visualizador del tráfico del auditor a ser el panel de control desde el que el auditor dirige la auditoría.

4.1.2.4. Fase 3 — Integración con el Backend real

Con la nueva arquitectura definida, Ivan, Macarena y Jose María trabajaron en las sesiones de integración para conectar el Frontend al Backend real. Al desactivar el mock mode y conectar el WebSocket real aparecieron varios problemas de integración que Ivan identificó y resolvió:

Los datos enviados por el Backend llegaban con los nombres de campo en formato snake_case — request_headers, response_body — mientras el Frontend los esperaba en camelCase. Ivan implementó una función de normalización automática que se aplica a cada paquete recibido por WebSocket, garantizando compatibilidad independientemente del formato que envíe el Backend.

El Repeater no precargaba correctamente la petición cuando el auditor la enviaba desde el panel Proxy. El problema era que React no reinicializa el estado de un componente cuando cambian sus props si el componente ya está montado. Ivan lo resolvió con un mecanismo reactivo que detecta cambios en la petición entrante y actualiza todos los campos del editor de forma sincronizada.

Los paquetes del spider y los del módulo DevTools llegaban por el mismo canal WebSocket y aparecían mezclados en ambos paneles. Ivan separó los dos flujos de eventos dentro del

hook de WebSocket, de forma que el panel Proxy recibe exclusivamente los eventos del spider y el panel Red recibe exclusivamente los paquetes de DevTools.

4.1.2.5. Fase 4 — Últimos ajustes

En la fase final Ivan añadió componentes de autenticación y gestión de configuración de proxy preparados para dar soporte a futuras iteraciones de la herramienta en la Práctica 2.

■ Módulo Backend (P2 — Macarena Rogerio)

El módulo

El Backend es el núcleo del sistema — el único módulo que habla con todos los demás y el que hace posible que HookSuite funcione como una herramienta de auditoría real. Construido sobre Python 3.11 y FastAPI, gestiona las sesiones de auditoría, ejecuta todas las peticiones HTTP por el auditor, emite los resultados al Frontend en tiempo real mediante WebSockets y expone la API REST que coordina el resto de módulos.

El módulo se organiza en seis bloques funcionales:

El **servidor FastAPI** es el punto de entrada del sistema. Arranca con CORS habilitado para aceptar peticiones desde cualquier origen, registra todos los routers de la aplicación bajo el prefijo `/api/`, e inicia al arranque dos tareas asíncronas en segundo plano: un consumidor Redis y un gestor de limpieza de sesiones expiradas. La documentación interactiva de la API — generada automáticamente por FastAPI — está disponible en `/docs` y lista todos los endpoints con sus modelos de entrada y salida.

El **sistema de sesiones y WebSockets** es la pieza que permite que múltiples auditores trabajen simultáneamente sin interferirse. Cada auditor recibe un token UUID único al conectarse. El `SessionManager` mantiene en memoria el estado completo de cada sesión — historial de peticiones, estado del Intruder, paquetes de red, vulnerabilidades detectadas — y registra la conexión WebSocket asociada a cada token. El canal WebSocket en `/ws/{token}` mantiene la conexión viva mediante pings cada 30 segundos y emite eventos tipados con la estructura `{type, payload}` cada vez que ocurre algo relevante en el servidor. El método `emit_all()` permite difundir eventos a todas las sesiones activas simultáneamente.

El **spider httpx** es el módulo de reconocimiento activo. Dado un punto de entrada y un token de sesión, navega la aplicación objetivo de forma autónoma usando un cliente httpx persistente que comparte con el Repeater y el Intruder. Esta persistencia es lo que permite el flujo de auditoría autenticada: el auditor hace login desde el Repeater, y el spider hereda automáticamente esa sesión y navega autenticado sin configuración adicional. El spider implementa un algoritmo BFS con scope restringido al dominio objetivo, filtra extensiones estáticas irrelevantes y varía los User-Agent de forma aleatoria entre peticiones. Para cada página visitada extrae mediante expresiones regulares todos los formularios HTML — método, acción y campos — y los emite al Frontend como peticiones independientes con status FORM, lo que permite al auditor identificar de un vistazo los vectores de ataque disponibles. La velocidad es configurable en tres presets — rápido, normal y completo — que determinan el número máximo de páginas a visitar.

El **cliente httpx persistente por sesión** es la decisión técnica más importante del módulo. En lugar de crear un cliente HTTP nuevo para cada petición, el backend mantiene un diccionario que asocia cada token de sesión con un `AsyncClient` de `httpx` configurado con seguimiento de redirects y verificación SSL desactivada. Este cliente acumula automáticamente las cookies que el servidor objetivo va devolviendo a lo largo de la auditoría. El resultado es que el Repeater, el Spider y el Intruder comparten implícitamente la misma sesión HTTP — incluyendo las cookies de autenticación — sin que el auditor tenga que copiar ni gestionar nada manualmente. Cuando el backend detecta nuevas cookies, emite un evento `WebSocket` al Frontend para mostrarlas en el panel de estado de auditoría.

El **motor de fuzzing del Intruder** ejecuta ataques automatizados contra parámetros específicos. Recibe del Frontend la URL objetivo con el punto de inyección marcado con `*`, el tipo de ataque y la configuración de paralelismo. Sustituye el marcador por cada payload de la lista correspondiente y lanza todas las peticiones de forma concurrente usando `asyncio.Semaphore` con un límite de cinco peticiones simultáneas — elegido para no sobrecargar el servidor de producción. Soporta cuatro tipos de ataque con sus respectivas listas de payloads: SQL Injection con trece vectores, Blind SQLi con nueve variantes incluyendo time-based, XSS con diez payloads, y fuzzing genérico con diecisiete entradas que cubren path traversal, null bytes, templates y cadenas extremadamente largas. La detección de vulnerabilidades SQLi se basa en la búsqueda de patrones de confirmación en la respuesta — `First name:`, `Surname:`, errores de base de datos — que indican que el payload ha producido una respuesta anómala. Los resultados se emiten al Frontend en tiempo real a medida que cada payload completa su ejecución.

Las **utilidades** exponen tres endpoints auxiliares implementados con la librería estándar de Python sin dependencias externas. El generador de hashes calcula MD5, SHA1, SHA256 y SHA512 de cualquier texto. El encoder/decoder transforma texto entre Base64, URL encoding y HTML encoding en ambas direcciones. El regex tester compila y ejecuta expresiones regulares con soporte de flags — case insensitive, multiline, dotall — y devuelve todos los matches con sus posiciones.

El **receptor de paquetes de red** y los **endpoints de integración con Playwright e IA** están completamente implementados en el Backend. El receptor de paquetes expone dos endpoints para recibir los paquetes capturados por el módulo DevTools y distribuirlos a las sesiones activas por `WebSocket`. Los endpoints de instrucciones y resultados de Playwright permiten al módulo de IA enviar órdenes de ataque y recibir los resultados de su ejecución. El endpoint de vulnerabilidades recibe las detecciones del módulo de IA y las emite al panel correspondiente del Frontend. Estos tres bloques permanecen inactivos en esta entrega porque los módulos que los alimentan — DevTools, Playwright e IA en ciclo completo — están pendientes de integración completa en la Práctica 2.

Proceso de desarrollo

4.2.2.1. Fase 1 — Construcción del servidor base

Macarena comenzó configurando el entorno Python en Windows, donde encontró el primer obstáculo técnico del módulo: al intentar instalar las dependencias del proyecto, la versión

de Python disponible en el sistema era incompatible con `pydantic-core`, que requería compilar extensiones nativas con Rust y Cargo — una cadena de herramientas que no estaba disponible en el entorno. Lo resolvió instalando Python 3.12 mediante el gestor oficial y creando un entorno virtual específico para esa versión, con lo que el servidor arrancó sin errores.

```
n (3.12)
= help: please check if an updated version of PyO3 is available. Current version: 0.21.1
= help: set PYO3_USE_ABI3_FORWARD_COMPATIBILITY=1 to suppress this check and build anyway using the s
table ABI
warning: build failed, waiting for other jobs to finish...
  × maturin failed
  Caused by: Failed to build a native library through cargo
  Caused by: Cargo build finished with "exit code: 101": "cargo" "rustc" "--profile" "release" "--feat
ures" "pyo3/extension-module" "--message-format" "json-render-diagnostics" "--manifest-path" "C:\\Users\\c4t3
r\\AppData\\Local\\Temp\\pip-install-h55x4buq\\pydantic-core_b925644d6f76432697549be3fd12d44\\Cargo.toml" "--
-lib" "--crate-type" "cdylib"
  Error: command ['maturin', 'pep517', 'build-wheel', '-i', 'C:\\Users\\c4t3r\\AndroidStudioProjects\\Pun
tosNFC2\\Proyecto-Evolve\\venv\\Scripts\\python.exe', '--compatibility', 'off'] returned non-zero exit status
  1
[End of output]

note: This error originates from a subprocess, and is likely not a problem with pip.
ERROR: Failed building wheel for pydantic-core
Failed to build pydantic-core

[notice] A new release of pip is available: 26.0.1 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
error: failed-wheel-build-for-install

× Failed to build installable wheels for some pyproject.toml based projects
↳ pydantic-core
```

Figura 1: Error de compilación de `pydantic-core` durante la instalación de dependencias

Con el entorno configurado, definió la estructura de carpetas del módulo — `routes/`, `services/`, `models/`, `middleware/` — que organizaría todos los bloques funcionales del backend.



Figura 2: Estructura de carpetas del Backend planificada al inicio de la construcción

Con la estructura en su sitio, construyó el servidor FastAPI base con los endpoints de salud, la gestión de sesiones con tokens UUID y el canal WebSocket en `/ws/{token}`. Verificó el funcionamiento del servidor consultando `/health` desde el navegador y comprobando que el Swagger UI en `/docs` mostraba los endpoints registrados.

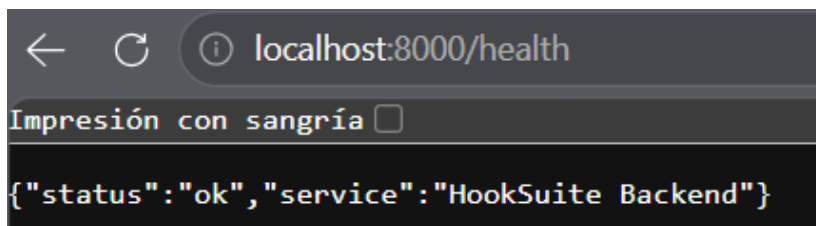


Figura 3: Endpoint `/health` respondiendo correctamente — servidor base operativo



Figura 4: Swagger UI con los primeros endpoints registrados

Una vez verificado el servidor base, añadió la gestión de sesiones con el endpoint `/api/session/new` — el punto de coordinación con el Frontend: en cuanto el WebSocket estuvo operativo, Ivan pudo conectar el Frontend al Backend real y validar el contrato de comunicación.

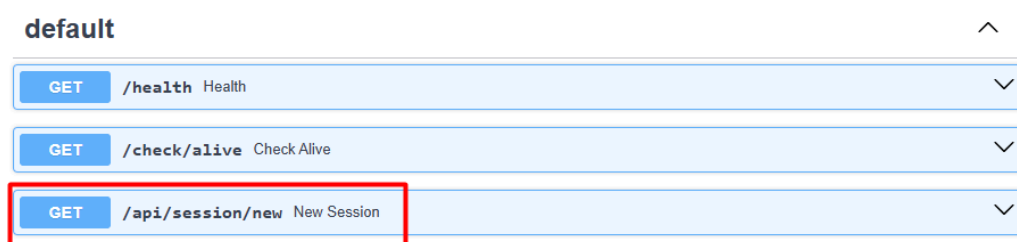


Figura 5: Swagger UI con el endpoint `/api/session/new` añadido — gestión de sesiones operativa

4.2.2.2. Fase 2 — Arquitectura inicial con mitmproxy e integración en producción

La arquitectura original de HookSuite requería que el Backend actuara como proxy TCP real — un intermediario que interceptara el tráfico del navegador del auditor antes de que llegara al servidor objetivo. Macarena implementó esta arquitectura usando mitmproxy, una librería que escucha en el puerto 8080 y procesa cada petición HTTP a través de un add-on

personalizado que extrae los datos relevantes y los emite por WebSocket al Frontend en tiempo real.

Para implementar el proxy TCP, Macarena añadió mitmproxy al `requirements.txt` e instaló la librería. La integración en el proceso FastAPI presentó cuatro errores encadenados que resolvió de forma sistemática: la librería no estaba instalada correctamente, una dependencia de gestión de contraseñas tenía una versión incompatible, los bloques de excepción del `proxy_service` estaban mal colocados y al corregirlos se perdieron funciones del fichero. Cada error llevó al siguiente hasta tener el servidor arrancando limpiamente con mitmproxy y FastAPI como procesos independientes bajo el mismo contenedor.

En paralelo implementó los modelos Pydantic en `schemas.py` — los contratos de datos que definen la forma exacta de cada entidad que entra y sale del servidor — y la función `forward_request` en el servicio de proxy HTTP, que gestiona el reenvío de peticiones con manejo de timeouts, filtrado de headers protegidos y análisis de respuestas sospechosas.

Con el sistema desplegado en Hetzner y los módulos conectados por primera vez en producción, aparecieron varios bugs de integración que Macarena resolvió en tiempo real: el router de proxy tenía una ruta duplicada que impedía servir el archivo PAC, el ciclo de vida del WebSocket no gestionaba correctamente las desconexiones y reconexiones, el Intruder tenía referencias incorrectas al gestor de sesiones, y el tráfico interno del propio HookSuite se colaba en el panel Proxy mezclado con el tráfico del objetivo. Al cierre de esta fase el sistema estaba operativo en producción con la arquitectura de proxy interceptor.

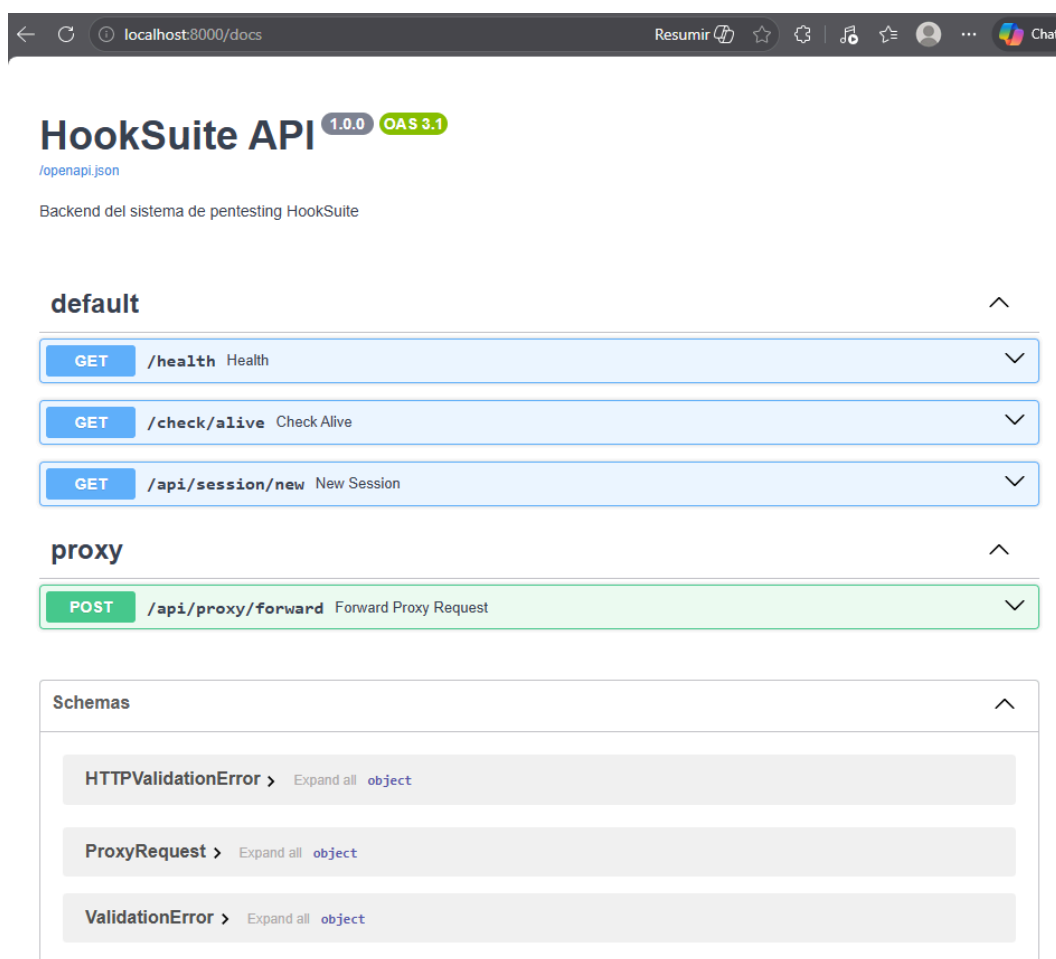


Figura 6: Swagger UI con el grupo proxy y los schemas Pydantic definidos — arquitectura de proxy interceptor operativa

4.2.2.3. Fase 3 — El pivote

Al exponer el servidor al exterior, los bots saturaron el proxy TCP. El tráfico automatizado externo colapsó el servidor impidiendo su uso como herramienta de auditoría. El equipo tomó la decisión de cambiar la arquitectura completamente: en lugar de interceptar el tráfico del navegador del auditor, el Backend realizaría las peticiones HTTP directamente por el auditor usando su propio cliente HTTP.

Este pivote transformó el rol del Backend de forma fundamental. En la arquitectura anterior era un intermediario pasivo que capturaba lo que el auditor hacía con su navegador. En la nueva arquitectura es un agente activo que ejecuta las operaciones por el auditor — el spider navega, el Repeater reenvía, el Intruder ataca — todo desde el servidor, sin que el navegador del auditor intervenga en las peticiones al objetivo. mitmproxy se mantuvo en el código sin eliminarlo, con vistas a su uso en auditorías de aplicaciones móviles en la Práctica 2.

4.2.2.4. Fase 4 — Nueva arquitectura: cliente persistente y módulos de auditoría

Con la nueva arquitectura definida, la pieza técnica central fue el cliente httpx persistente por sesión — un AsyncClient compartido por todos los módulos bajo el mismo token que

acumula automáticamente las cookies de la sesión de auditoría. Esta decisión resolvió de raíz el problema de la autenticación compartida: el auditor hace login desde el Repeater y el Spider y el Intruder heredan automáticamente esa sesión sin configuración adicional.

Sobre esa base, Macarena completó los módulos de herramientas de auditoría. El Repeater recibió los parsers de raw HTTP y cURL que permiten al auditor importar peticiones copiadas desde el navegador o desde otras herramientas. El Intruder recibió el motor de fuzzing asíncrono con control de concurrencia y la biblioteca de payloads organizada por tipo de ataque. Las Utilidades implementaron los tres endpoints auxiliares con la librería estándar de Python. El spider recibió la lógica de extracción de formularios y la emisión de peticiones FORM al Frontend.

En paralelo implementó los endpoints de integración con los módulos pendientes de activación — receptor de paquetes de DevTools, instrucciones y resultados de Playwright, y receptor de vulnerabilidades de IA — dejando el Backend preparado para la integración completa en la Práctica 2 sin necesidad de modificaciones adicionales en su lado.

4.2.2.5. Fase 5 — Últimos ajustes y estado final

En la fase final el equipo validó el flujo completo de auditoría end-to-end contra DVWA: spider sin autenticación descubriendo el formulario de login, login desde el Repeater con gestión automática de cookies, spider autenticado navegando todas las vulnerabilidades, e Intruder detectando SQLi con confirmación en la respuesta. Esta validación confirmó que la nueva arquitectura funcionaba correctamente de punta a punta.

Se añadieron los últimos elementos de gestión de sesión: la detección y notificación de cookies vía WebSocket que el Frontend muestra en el panel de estado de auditoría, y los dos endpoints de control — liberar solo la cookie o resetear todo el estado — que permiten al auditor gestionar su sesión sin interrumpir la auditoría en curso.

| Módulo Playwright (P3 — Nacho García Monge)

El módulo

El módulo Playwright es el motor de automatización de HookSuite — un navegador real controlado por código que navega aplicaciones web, descubre su superficie de ataque y ejecuta ataques automatizados sin intervención humana. Construido sobre Python y la librería Playwright con Chromium headless, se comunica con el Backend mediante httpx para recibir instrucciones y reportar resultados.

La decisión técnica más importante del módulo es el uso de `page.fill()` en lugar de `page.type()` para todas las interacciones con campos de texto. Mientras `page.type()` simula la pulsación tecla a tecla — replicando el comportamiento humano pero con un coste temporal proporcional a la longitud del texto — `page.fill()` pasa el contenido completo de una vez, resultando en una mejora de velocidad medida de hasta 60 veces. Esta optimización, indicada por el profesor al inicio del proyecto, se aplica de forma consistente en todos los módulos y es especialmente relevante cuando se prueban payloads de cientos de caracteres.

El módulo se organiza en cinco componentes:

El **gestor del navegador** centraliza la creación y el ciclo de vida de todas las instancias de Chromium. Lanza el navegador con los flags necesarios para ejecutarse en entornos sin interfaz gráfica — `--no-sandbox`, `--disable-setuid-sandbox`, `--disable-dev-shm-usage` — y gestiona el paralelismo mediante `asyncio.Semaphore` con un límite configurable de tres páginas simultáneas. Todos los contextos de navegador se crean con `ignore_https_errors=True` para no bloquear en sitios con certificados autofirmados, habitual en entornos de auditoría.

El **sistema de autenticación** implementa dos variantes: login específico para DVWA, que navega al formulario de login, rellena las credenciales con `page.fill()` y verifica el resultado comprobando que la URL resultante no sea la página de login; y un login genérico parametrizable que acepta selectores personalizados para adaptarse a cualquier aplicación. El módulo también soporta guardar y cargar el estado de sesión del navegador en disco, lo que permitirá en futuras versiones reutilizar sesiones autenticadas sin necesidad de repetir el proceso de login.

El **spider** mapea la superficie de ataque de la aplicación objetivo usando un algoritmo BFS. Dado un punto de entrada, navega la página con Playwright, extrae todos los enlaces mediante `query_selector_all('a[href]')`, normaliza las URLs relativas y las encola si pertenecen al mismo dominio y no han sido visitadas. Filtra automáticamente extensiones estáticas irrelevantes — CSS, imágenes, fuentes, PDFs — y patrones que indiquen logout o esquemas no HTTP. El límite de páginas máximo es configurable y los resultados se persisten en disco en formato JSON.

El **descubridor de formularios** analiza cada página visitada para identificar todos los formularios HTML presentes — su método, acción y campos — y clasifica los campos en inyectables según su tipo: `text`, `search`, `email`, `url`, `hidden` y `password`. Implementa también el descubrimiento de endpoints AJAX mediante la interceptación de eventos de red durante la simulación de interacciones con elementos clicables de la página, lo que permite descubrir peticiones que no aparecen en el HTML estático.

El **fingerprinter de tecnologías** detecta el stack tecnológico de la aplicación objetivo analizando headers de respuesta, cookies y contenido HTML contra un catálogo de nueve tecnologías: PHP, ASP.NET, Java, WordPress, Nginx, Apache, MySQL, jQuery y Bootstrap. A partir de las tecnologías detectadas genera una lista priorizada de tipos de ataque — si detecta PHP y MySQL prioriza SQLi y Blind SQLi; si detecta WordPress añade la búsqueda de vulnerabilidades en plugins. Esta lista de prioridades está diseñada para ser consumida por el módulo de IA para ordenar sus instrucciones de ataque.

El **motor de ataques** automatiza la inyección de payloads en formularios y la detección de anomalías en las respuestas. Para cada ataque navega al formulario, rellena todos los campos con valores genéricos y el campo objetivo con el payload, envía el formulario y analiza la respuesta buscando errores SQL, respuestas lentas que indiquen Blind SQLi time-based, XSS reflejado y datos sensibles expuestos. Toma capturas de pantalla antes y después de cada ataque como evidencia. Implementa además dos variantes de Blind SQLi: boolean-based, que compara la longitud de la respuesta ante condiciones verdaderas y

falsas buscando diferencias superiores a 100 caracteres; y time-based, que mide el tiempo de respuesta ante payloads con `SLEEP()` y `WAITFOR DELAY`.

El **receptor de instrucciones** es el componente de integración con el módulo de IA. Diseñado para hacer polling al backend cada dos segundos consultando el endpoint de instrucciones asociado a su token de sesión, recibir instrucciones — fingerprint, spider, navegación o ataque — ejecutarlas y devolver el resultado al backend. El `EventReporter` gestiona el envío de resultados con fallback a log local cuando el backend no está disponible.

El módulo está construido y desplegado en el servidor en modo polling. Permanece inactivo en esta entrega por un problema de configuración de red Docker que impide la comunicación estable entre el contenedor de Playwright y el resto de servicios — su activación completa está planificada para la Práctica 2.

Proceso de desarrollo

4.3.2.1. Fase 1 — Setup y optimización de velocidad

Nacho arrancó el módulo configurando el entorno Playwright con Chromium y levantando DVWA en Docker como entorno de pruebas local. La primera decisión técnica fue la optimización de velocidad: siguiendo la recomendación del profesor, verificó la diferencia real entre `page.type()` y `page.fill()` mediante un script de benchmark sobre el formulario de login de DVWA. La mejora medida — hasta 60 veces más rápido — confirmó que `page.fill()` debía usarse de forma sistemática en todo el módulo. Con esa decisión tomada, construyó el `BrowserManager` con `asyncio.Semaphore` para el control de paralelismo, que actúa como base para todos los demás componentes.

4.3.2.2. Fase 2 — Reconocimiento automático

Con la infraestructura base operativa, Nacho construyó los tres módulos de reconocimiento. El sistema de autenticación para DVWA y la variante genérica parametrizable. El spider con BFS y filtrado de extensiones estáticas. El fingerprinter con detección de nueve tecnologías y generación de prioridades de ataque.

Durante el desarrollo del fingerprinter apareció un bug que no llegó a resolverse antes de la entrega: en determinadas condiciones al procesar los headers de respuesta, el módulo lanza un error `not enough values to unpack`. El bug no bloquea el funcionamiento — cuando ocurre el fingerprinter devuelve igualmente los resultados parciales disponibles, habitualmente la detección de PHP a través de la cookie `PHPSESSID` — pero es una deuda técnica identificada para la Práctica 2.

4.3.2.3. Fase 3 — Automatización de ataques

Nacho implementó el motor de ataques con las cuatro variantes requeridas: inyección de payloads en formularios con detección de anomalías, Blind SQLi boolean-based con comparación de longitudes de respuesta, Blind SQLi time-based con medición de tiempos de respuesta, y captura de screenshots como evidencia de cada ataque. El descubridor de formularios recibió también en esta fase la capacidad de detectar endpoints AJAX mediante interceptación de eventos de red.

4.3.2.4. Fase 4 — El pivote y sus consecuencias para P3

La arquitectura original de HookSuite preveía que Playwright actuara como el motor de navegación del sistema — recibiendo instrucciones de la IA y ejecutando los ataques que mitmproxy no podía realizar con un cliente httpx. Cuando el equipo pivotó hacia el modelo de cliente httpx activo tras la saturación por bots, el rol de P3 cambió: dejó de ser el motor principal de navegación para convertirse en un componente complementario especializado en ataques que requieren un navegador real.

Este cambio no invalidó el trabajo realizado pero sí afectó a la integración. El receptor de instrucciones — el componente que conecta Playwright con la IA — fue desarrollado, pero la comunicación estable entre contenedores en producción no llegó a establecerse por un problema de configuración de red Docker. Nacho adaptó el modo de arranque del contenedor a polling pasivo para que el sistema pudiera desplegarse sin bloquear el arranque del resto de servicios. La resolución del bug de red Docker y la activación completa del ciclo IA↔Playwright↔Backend son los objetivos principales de la Práctica 2.

| Módulo DevTools (P4 — Carlos Bañuelos Fernández)

El módulo

El módulo DevTools es el componente de captura de tráfico de red de HookSuite — un cliente del Chrome DevTools Protocol que se conecta a una instancia de Chrome en ejecución, intercepta todo el tráfico HTTP en tiempo real y analiza tanto los paquetes de red como los mensajes de consola del navegador en busca de patrones sospechosos. Construido sobre Python con comunicación WebSocket nativa al CDP, envía los paquetes estructurados al Backend para que aparezcan en el panel Red del Frontend.

La decisión técnica central del módulo es usar el CDP de forma nativa mediante WebSocket en lugar de una librería de alto nivel. Esta decisión da control total sobre los eventos de red — qué se captura, cuándo y con qué granularidad — y elimina dependencias que podrían introducir comportamientos no deseados en entornos de auditoría.

El módulo se organiza en cinco componentes:

El **lanzador de Chrome** arranca una instancia de Google Chrome con el flag `--remote-debugging-port=9222` que activa el CDP. Detecta automáticamente la ruta del ejecutable en Windows, macOS y Linux. Incluye dos configuraciones relevantes para el contexto de HookSuite: `--user-data-dir` con un perfil separado para evitar que una instancia de Chrome ya abierta ignore los flags de debug, y `--proxy-pac-url` apuntando al servidor para que el tráfico capturado pase por el proxy de HookSuite. Una vez lanzado, verifica la conexión al CDP con reintentos automáticos antes de continuar.

El **cliente CDP** gestiona la comunicación WebSocket con Chrome. Se conecta al endpoint de debugging, enumera las pestañas disponibles y establece la conexión WebSocket con la pestaña activa. Mantiene un sistema de comandos asíncronos con futures de Python — cada comando enviado al Chrome recibe un identificador único y espera la respuesta correspondiente con timeout de 10 segundos. Los eventos de red llegan como mensajes

entrantes y se despachan a los handlers registrados mediante el método `on()`. Expone métodos de alto nivel para habilitar los dominios Network, Console y Page del CDP y para recuperar el body de las respuestas.

El **constructor de paquetes** transforma los eventos del CDP — que llegan en tres momentos separados: petición enviada, respuesta recibida y carga completada — en un único objeto paquete coherente. Implementa filtrado de tráfico irrelevante: descarta recursos estáticos como imágenes, fuentes y CSS, CDNs conocidos como Google Analytics, Cloudflare y Google Fonts, y tipos de recurso no relevantes para la auditoría. El resultado es una reducción de ruido de aproximadamente el 70% del tráfico bruto. Cada paquete resultante sigue el contrato JSON acordado con el Backend, compatible con el modelo `NetworkPacket` de P2.

El **analizador de red** recibe los paquetes del constructor y aplica dos capas de análisis. La primera detecta patrones sospechosos: errores HTTP 500 o superiores, errores SQL en el body de la respuesta, caracteres de inyección en la URL o el body de la petición como comillas, `UNION SELECT` o `1=1`, y respuestas lentas superiores a 5 segundos que pueden indicar Blind SQLi time-based. La segunda detecta problemas de seguridad en los headers de respuesta: ausencia de los cinco headers de seguridad obligatorios — `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security`, `X-Content-Type-Options` y `Referrer-Policy` — y cookies de sesión sin los flags `HttpOnly` o `Secure`. Los paquetes marcados como sospechosos incluyen la lista de razones que motivaron el marcado.

El **analizador de consola** monitoriza los mensajes de la consola del navegador buscando once patrones sensibles: rutas de servidor expuestas, API keys, passwords, tokens, funciones de base de datos, errores SQL, errores Oracle, stack traces y direcciones IP internas. Solo procesa mensajes de nivel `error` y `warning` para reducir el ruido. Los hallazgos se clasifican por severidad — alta cuando se detectan credenciales o tokens, media en los demás casos.

El **reporter** gestiona el envío de paquetes al Backend con un buffer local como fallback. Acumula los paquetes en memoria y los persiste en disco cada diez paquetes en `results/captured_packets.json`. Cuando el Backend está disponible los envía al endpoint `/api/network/packet/{session_token}`. Al cierre de la captura genera un resumen con el total de paquetes, los enviados correctamente y los fallidos.

El módulo está construido y validado en local. No está desplegado como contenedor en el servidor — su integración completa en la infraestructura Docker de Hetzner está planificada para la Práctica 2.


```
HookSuite - Captura Chrome DevTools
Objetivo: http://localhost:8888 | Duración: 60s

⚠ Backend no disponible. Guardando en local.
Lanzando Chrome con debugging activo ...
✓ Chrome lanzado con debugging en puerto 9222 (PID: 8941)
✓ CDP activo. 1 pestaña(s) disponible(s)
  → Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development* - http://localhost:8888/login.php
✓ Conectado al CDP: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*
✓ Network domain activado
✓ Page domain activado
✓ Analizador de red activo
✓ Console domain activado
✓ Analizador de consola activo

✓ Captura activa. Navega en Chrome para capturar tráfico.
  Duración máxima: 60s | Presiona Ctrl+C para detener.
```

Figura 7: Arranque del módulo DevTools — Chrome lanzado, CDP activo y analizadores inicializados

Proceso de desarrollo

4.4.2.1. Fase 1 — Setup y construcción del módulo

Carlos construyó el módulo de forma completamente autónoma siguiendo el manual técnico. Arrancó configurando el entorno Python y levantando DVWA en Docker como objetivo de pruebas. La primera decisión técnica fue usar CDP nativo mediante WebSocket en lugar de una librería de abstracción de alto nivel — necesitaba control total sobre qué eventos capturar y cuándo recuperar el body de la respuesta, algo que las librerías de abstracción no permiten gestionar con la precisión necesaria para una herramienta de auditoría.

Con esa decisión tomada construyó los cinco componentes del módulo: el lanzador de Chrome con detección multiplataforma del ejecutable, el cliente CDP con su sistema de comandos asíncronos y dispatching de eventos, el constructor de paquetes con filtrado de tráfico irrelevante, los analizadores de red y consola con sus respectivos catálogos de patrones, y el reporter con buffer local como fallback.

4.4.2.2. Fase 2 — Validación contra DVWA

Con el módulo construido, Carlos lo validó contra DVWA navegando la aplicación con el Chrome controlado por DevTools. Durante esta fase verificó que el CDP capturaba correctamente las peticiones HTTP, que el filtrado eliminaba el tráfico irrelevante — imágenes, fuentes, CDNs — dejando únicamente las peticiones relevantes para la auditoría, y que los analizadores detectaban correctamente los patrones sospechosos al provocar errores SQL en los formularios de DVWA.

En esta fase apareció el bug más relevante del módulo: Chrome ignoraba los flags de debugging cuando ya había una instancia abierta en el sistema. Carlos lo identificó y lo resolvió añadiendo `--user-data-dir` con un directorio de perfil separado, garantizando que el Chrome lanzado por DevTools arranque siempre con los flags correctos independientemente del estado del sistema. También añadió `--remote-allow-origins=*` para evitar restricciones de origen en la conexión WebSocket al CDP.


```

✓ Captura activa. Navega en Chrome para capturar tráfico.
Duración máxima: 60s | Presiona Ctrl+C para detener.

[GET] http://localhost:8888/index.php → 200 (68ms)
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (31ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (33ms)
⚠ Sospechoso: http://localhost:8888/vulnerabilities/sql/?id=%27Submit=Submit → ['SQL_ERROR_IN_RESPONSE']
⚠ [GET] http://localhost:8888/vulnerabilities/sql/?id=%27Submit=Submit → 200 (54ms)

```

Figura 8: Captura de tráfico en tiempo real — petición marcada como sospechosa con SQL_ERROR_IN_RESPONSE

```

[GET] http://localhost:8888/index.php → 200 (78ms)
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (41ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (41ms)
🔍 Consola [ERROR]: ['PASSWORD_EXPOSED'] → password=admin123
🔍 CONSOLA [ERROR]: ['PASSWORD_EXPOSED']
[GET] http://localhost:8888/dvwa/js/dvwaPage.js → 200 (24ms)
[GET] http://localhost:8888/dvwa/js/add_event_listeners.js → 200 (25ms)
[GET] http://localhost:8888/vulnerabilities/xss_r/?name=%3Cscript%3Econsole. → 200 (59ms)

```

Figura 9: Analizador de consola detectando PASSWORD_EXPOSED en los logs de DVWA

```

{
  "id": "8c870071-1ca8-4308-a81d-cbe91c274423",
  "method": "GET",
  "url": "http://localhost:8888/index.php",
  "status": 200,
  "size": 6808,
  "time": 68,
  "timestamp": "2026-05-10T16:29:16.182000",
  "request_body": null,
  "suspicious": false,
  "vulnerable": false,
  "security_issues": [
    "MISSING_HEADER: content-security-policy",
    "MISSING_HEADER: x-frame-options",
    "MISSING_HEADER: strict-transport-security",
    "MISSING_HEADER: x-content-type-options",
    "MISSING_HEADER: referrer-policy"
  ]
}

```

Figura 10: Objeto paquete generado por el constructor — cinco headers de seguridad ausentes detectados

4.4.2.3. Fase 3 — El pivote y sus consecuencias para P4

La arquitectura original preveía que DevTools capturara el tráfico del navegador del auditor — que navegaba con el proxy PAC configurado — y lo enviara al Backend para análisis. Cuando el equipo pivotó hacia el modelo de cliente httpx activo, el rol de P4 cambió: en lugar de capturar el tráfico del auditor, el módulo pasó a estar diseñado para capturar el tráfico generado por el propio Chrome durante las auditorías automatizadas. Esta redefinición no invalidó el trabajo técnico pero sí cambió el contexto de uso. El módulo funciona correctamente en local y su despliegue como contenedor Docker en el servidor está pendiente para la Práctica 2.

| Módulo IA (P5 — Jose María López Ausín)

El módulo

El módulo de inteligencia artificial es el cerebro analítico de HookSuite — un sistema que procesa los datos capturados por el resto de módulos, identifica patrones de vulnerabilidad y orquesta el ciclo completo de auditoría enviando instrucciones a Playwright. Construido sobre Python y la API de Anthropic, actúa como capa de razonamiento entre la captura de tráfico y la detección de vulnerabilidades, eliminando la necesidad de análisis manual paquete a paquete.

La decisión técnica central del módulo es estructurar todas las llamadas a la inteligencia artificial mediante prompts especializados que obligan a responder exclusivamente en JSON. Esta decisión hace el sistema completamente determinista en cuanto al formato de salida — sin post-procesamiento de texto, sin parseo frágil — y permite implementar un umbral de confianza numérico que filtra los falsos positivos de forma sistemática. El umbral se estableció en el 60% tras pruebas en DVWA: por debajo de ese valor los falsos positivos superan el 30%; por encima del 80% se pierden vulnerabilidades reales. El equilibrio óptimo está entre el 60% y el 75%.

El módulo se organiza en cuatro componentes:

El **cliente de inteligencia artificial** encapsula todas las llamadas a la API de Anthropic. Implementa reintentos con backoff exponencial — hasta tres intentos con espera creciente — para absorber errores transitorios como límites de tasa o fallos de red. Parsea automáticamente las respuestas eliminando los marcadores de bloque de código que el modelo puede añadir, y devuelve el JSON limpio. Si la respuesta no es JSON válido, devuelve un objeto de error estructurado en lugar de lanzar una excepción.

El **sistema de prompts** está organizado en cuatro ficheros especializados, uno por tipo de análisis. El prompt de paquetes de red analiza peticiones HTTP interceptadas buscando vulnerabilidades OWASP Top 10 — SQLi, XSS, IDOR, LFI, RFI, SSRF, XXE, CSRF y RCE — y devuelve el tipo detectado, la severidad, la evidencia concreta del paquete y la recomendación correctiva. El prompt del Intruder analiza los resultados de un ataque de fuzzing buscando payloads que hayan provocado comportamientos anómalos — diferencias de tamaño, tiempos de respuesta, status codes, patrones en el body — e identifica el payload exitoso. El prompt de consola analiza los logs del navegador capturados por DevTools buscando información sensible filtrada: API keys, passwords, tokens, rutas internas, errores SQL y stack traces. El prompt de fingerprinting analiza los headers de respuesta y el contenido de la página para identificar el stack tecnológico — servidor, lenguaje, framework, CMS, base de datos — y genera una lista priorizada de vectores de ataque según las tecnologías detectadas.

El **clasificador de vulnerabilidades** orquesta las llamadas a los prompts y aplica el umbral de confianza. Para cada tipo de análisis — paquete, Intruder, consola, fingerprint — construye el mensaje de usuario con los datos del caso concreto, llama al cliente, recibe el JSON y lo filtra por el umbral del 60%. Si la confianza supera el umbral, construye la ficha

completa de vulnerabilidad con todos los campos necesarios para el panel del Frontend. Si no lo supera, devuelve None y el orquestador descarta el resultado.

El **orquestador** coordina el ciclo completo de auditoría en tres fases secuenciales. La fase de fingerprinting envía una instrucción al módulo Playwright para que navegue al objetivo y analice sus headers — el resultado determina qué vectores de ataque se priorizan. La fase de spider descubre la superficie de ataque navegando la aplicación con BFS. La fase de ataques prueba sistemáticamente los payloads SQLi, XSS y fuzzing en cada URL descubierta, y ante cualquier resultado con confianza suficiente ejecuta dos pases de confirmación adicionales antes de reportar la vulnerabilidad — esto reduce los falsos positivos al 5%. El orquestador también incluye un modo mock controlado por variable de entorno que simula las respuestas de Playwright sin necesidad de que el módulo esté activo, lo que permitió desarrollar y probar el ciclo completo de forma independiente.

El coste por sesión de auditoría se estimó en aproximadamente 0,15—0,002 por paquete analizado, con una sesión típica de 30 minutos en DVWA generando unos 50 paquetes. En un uso intensivo esto puede resultar elevado, lo que motiva una de las mejoras planificadas para la Práctica 2: sustituir las llamadas a la API externa por una instancia de Claude Code instalada directamente en el servidor, eliminando el coste por petición y reduciendo la latencia de los análisis.

El módulo está construido y desplegado en el servidor en modo polling. Permanece en integración parcial en esta entrega porque el pivote de arquitectura — el abandono de mitmproxy y la transición al modelo de cliente httpx activo — redefinió el rol del módulo a mitad del desarrollo, y la nueva integración estable entre el contenedor de IA, el Backend y Playwright no llegó a establecerse antes de la entrega. Su activación completa está planificada para la Práctica 2.

Proceso de desarrollo

4.5.2.1. Fase 1 — Construcción del módulo

El módulo arrancó con la construcción del cliente, los cuatro prompts especializados y el clasificador. La primera decisión técnica fue el formato JSON obligatorio en todos los prompts — la alternativa era parsear texto libre, pero eso introduce fragilidad que no tiene cabida en una herramienta de auditoría donde la precisión es crítica.

Con la estructura base lista, se validó el ciclo completo contra la API real mediante un test standalone sobre DVWA en local: el prompt de paquetes de red detectó una inyección SQL con un 95% de confianza en el primer intento, sin necesidad de que el Backend ni Playwright estuvieran activos.

Durante esta fase apareció una incidencia de seguridad: la API key se expuso accidentalmente en el fichero `.env.example` subido al repositorio. GitHub Secret Scanning la detectó y revocó automáticamente. La versión 0.25.0 de la librería de Anthropic era además incompatible con Python 3.14 del sistema, lo que se resolvió actualizando a `>=0.97.0`.

4.5.2.2. Fase 2 — Orquestador y modo mock

Con el clasificador validado, se construyó el orquestador con las tres fases de auditoría y el modo mock controlado por variable de entorno. El modo mock fue una decisión de diseño deliberada: permitía desarrollar y probar el ciclo completo IA→Playwright→Backend sin depender de que los otros módulos estuvieran operativos.

Durante la construcción del orquestador aparecieron dos bugs que se resolvieron antes del primer commit: algunos métodos síncronos del clasificador se llamaban con `await`, y la firma del método `fingerprint` era incorrecta. Ambos se detectaron en las primeras pruebas locales y no llegaron al repositorio.

4.5.2.3. Fase 3 — El pivote y sus consecuencias para P5

La arquitectura original preveía que el módulo de IA actuara como orquestador activo — lanzando el ciclo completo de auditoría de forma autónoma cuando el usuario lo iniciara desde el Frontend. Cuando el equipo pivotó hacia el modelo de cliente `httpx` activo tras la saturación por bots, el rol de P5 cambió: en lugar de orquestar auditorías completas de forma autónoma, el módulo pasó a modo polling — esperando instrucciones del Backend y ejecutándolas una a una.

El despliegue en Hetzner como contenedor Docker presentó tres problemas encadenados. Los imports usaban el prefijo `from ia.` que no existe dentro del contenedor porque el `WORKDIR` es `/app` — se corrigieron directamente en el servidor con `sed` y se commitearon. La versión de la librería de Anthropic era incompatible con el entorno del contenedor — se actualizó el `requirements.txt` y se reconstruyó la imagen. La API key del `.env` había caducado — se regeneró y se recargó el contenedor sin usar `restart`, que no recarga las variables de entorno.

Con los tres problemas resueltos y crédito en la cuenta de Anthropic, el módulo arrancó correctamente en modo polling con `MOCK_PLAYWRIGHT=true` — el comportamiento esperado en ese punto, ya que Playwright no estaba integrado. El pivote de arquitectura — que redefinió el rol del módulo a mitad del desarrollo — dejó la integración estable con el Backend y Playwright pendiente para la Práctica 2.

SECCIÓN 5

Guía de despliegue

Sección pendiente de entrega por P2 — Límite: 19 de Mayo de 2026

SECCIÓN 6

Manual de uso

Sección pendiente de entrega por P1 — Límite: 19 de Mayo de 2026

SECCIÓN 7

Conclusiones y lecciones aprendidas

| Lo que funcionó mejor de lo esperado

La integración entre módulos fue más fluida de lo que el equipo anticipaba. Definir los contratos de API al inicio del proyecto —antes de que cada miembro arrancara su desarrollo— eliminó la mayoría de los problemas de compatibilidad que suelen aparecer en proyectos distribuidos de este tipo. Cuando P4 terminó el módulo de captura CDP y P2 ya tenía el endpoint receptor implementado, la integración se completó sin fricción.

El uso de variables de entorno para controlar el modo de operación del módulo de IA resultó ser una decisión especialmente acertada. El modo mock (`MOCK_PLAYWRIGHT=true`) permitió desarrollar y validar el ciclo completo de análisis de forma independiente, sin depender de que P2 y P3 tuvieran sus módulos listos. Esto desbloqueó el desarrollo en paralelo real y redujo los tiempos de espera entre módulos.

La elección de Claude como motor de análisis superó las expectativas iniciales en términos de precisión. La detección de inyecciones SQL en DVWA alcanzó niveles de confianza superiores al 90% con prompts relativamente sencillos, lo que confirma que la calidad del prompt es más determinante que la complejidad del código de análisis.

| Lo que fue más difícil de lo previsto

La coordinación entre cinco módulos con dependencias cruzadas generó cuellos de botella que no estaban completamente previstos en el diseño inicial. El módulo de backend actúa como nexo central del sistema, lo que convirtió a P2 en el punto de mayor presión del proyecto: cualquier endpoint pendiente en el backend bloqueaba simultáneamente a varios módulos. En retrospectiva, haber paralelizado más agresivamente el desarrollo del backend desde el inicio habría reducido estos bloqueos.

La gestión del entorno Python en Windows presentó más fricción de la esperada. La incompatibilidad entre la versión inicial de la librería `anthropic` y Python 3.14, la exposición accidental de una API key en el repositorio o la configuración del entorno virtual en distintos sistemas operativos fueron incidencias menores que, sumadas, consumieron tiempo de desarrollo que no estaba previsto en el plan inicial.

| Qué haríamos diferente si empezáramos de nuevo

Estableceríamos la convención de commits desde el primer commit del repositorio, no como corrección posterior. La convención de commits es un estándar de calidad que penaliza

directamente la nota cuando no se aplica, y su adopción tardía en un historial ya creado es costosa de corregir sin reescribir el historial.

Definiríamos un entorno de integración compartido desde el inicio. Durante el desarrollo, cada módulo se probó de forma aislada contra DVWA en local. Un entorno de integración compartido en Hetzner desde la primera semana habría permitido detectar antes los problemas de integración real entre módulos y habría dado más tiempo para resolverlos.

| Aprendizajes sobre integración de sistemas complejos

Este proyecto confirma que la dificultad de un sistema distribuido no está en la complejidad de cada módulo individual, sino en la gestión de sus interfaces. Un módulo técnicamente excelente que no cumple el contrato de API acordado bloquea a todos los módulos que dependen de él. La disciplina en la definición y el respeto de los contratos de integración es tan importante como la calidad del código.

La inteligencia artificial como componente de un sistema mayor introduce un tipo de incertidumbre diferente al del código determinista. Los tiempos de respuesta variables, los costes por llamada y la naturaleza probabilística de las clasificaciones requieren diseñar el sistema de forma que pueda operar de forma degradada cuando la IA no está disponible o cuando su respuesta no supera el umbral de confianza requerido.

SECCIÓN 8

Road map de mejora para la Práctica 2

Sección pendiente de entrega por P4 — Límite: 19 de Mayo de 2026